In chapter 6, you learned how to use a signal to activate side effects when a `User` object is created. Django also provides signals for other things, including login failure. The principle is similar; you use a function decorated with `@receiver` to register it to receive the signal, and you can access information about the event through the function's keyword arguments. The following listing shows a function that traps the login-failed signal and prints a message.

Listing E.23 Invoke a side effect when a login failure occurs

```
# RiffMates/bands/models.py
from django.contrib.auth.signals import user_login_failed
from django.dispatch import receiver
...

@receiver(user_login_failed)
def track_login_failure(sender, **kwargs):
    username = kwargs["credentials"]["username"]
    url = kwargs["request"].path

    print(f"LOGIN Failure by {username} for {url}")
```

The best place for this signal handler isn't obvious; I've arbitrarily chosen to put it with the other signal handler in the code.

The user_login_failed signal is invoked when a username/password mismatch occurs.

The user_login_failed signal includes credential information and the request object in its keyword argument dictionary.

A function registers itself to handle a signal with the @receiver decorator, which requires a reference to the signal object.

E.6 Chapter 7

The web is a much more interesting and dynamic place when you handle input from your users. Chapter 7 showed you how to process forms and deal with uploaded files.

Exercise 1

Modify the `musician` view to display an optional description and bio-pic. Add an `edit_musician()` view, so users can self-serve the creation and editing of their musician profiles. Insert an Add Musician Profile link to the home page and an Edit link to the Musician Details page if the logged-in user is the owner.

Like with the addition of a description and picture to the `Venue` class in chapter 7, this first requires new fields on the `Musician` class, shown in the following listing.

Listing E.24 New fields for the musician model

```
# RiffMates/bands/models.py
...
class Musician(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
```